AI4Research Repository QA Control Pack

Coding-agent instructions for repo reference, side-effect policy, evidence storage, report template, and execution prompt

Purpose. Use this document together with the updated QA execution workbook. The workbook contains feature IDs, function inventory templates, entrypoint mapping, existing/missing test mapping, pass/fail criteria, and severity rubric. This document defines the operating constraints and final report requirements for the coding agent.

1. Exact repository reference

Requirement. The coding agent must test an immutable repo state, not an ambiguous moving branch. Before testing, it must record the repo URL, branch, and exact commit SHA in both the evidence directory and final report.

Field	Value to use / fill
Repo	https://github.com/Stellven/AI4Research
Branch	openJiuwen-Solar
Commit SHA	<agent must fill using git rev-parse HEAD>
Mode	audit/test only unless explicitly authorized
Primary taxonomy	ai4research_recursive_feature_split_qa_execution.xlsx
Atomic feature rule	Level 4 is the intended atomic feature; if a row has no Level 4, use the deepest non-empty level.

```
git status --short
git rev-parse --show-toplevel
git branch --show-current
git rev-parse HEAD
git remote -v
```

2. Environment and side-effect policy

Default stance. Tests should be deterministic and isolated by default. The agent must not use real credentials, mutate real user state, publish releases, send email, run remote jobs, or perform destructive mutations unless the user gives explicit approval for a separate live phase.

Side-effect class	Default policy	Allowed only when
Read local files	Allowed in isolated checkout	Path is inside repo or test temp directory
Write artifacts	Allowed	Under the designated evidence directory or isolated test temp root
Wiki/local mutation	Blocked unless test explicitly verifies bounded mutation	Approval artifact or parity_demo/local fixture mode exists
Network/provider fetch	Blocked by default	Live-provider phase is explicitly approved and recorded
Remote execution	Blocked by default	Explicit approval, allowlist, before/runtime/after artifacts
Email send	Blocked by default	Explicit approval and test mailbox/sandbox
Credential/config mutation	Blocked by default	Explicit user-provided values and approval
Destructive reset/delete	Blocked by default	Dry-run first, explicit approval, isolated target only
Release/tag/push/upload	Blocked	Separate release authorization only

- Run installer and harness tests in isolated HOME/SOLAR_HOME/CLAUDE_DIR directories, never against the user's real ~/.solar or ~/.claude.
- A gated route may pass by returning BLOCKED_EXPECTED or INCONCLUSIVE_EXPECTED with a typed continuation/access request.
- No test may treat fixture or smoke evidence as live-provider/full-parity evidence.
- Every blocked or skipped test must state the exact missing approval, credential, tool, platform, or provider.

3. Evidence storage convention

All test evidence must be durable. The agent should create one timestamped evidence directory and write every command, output, generated artifact, and feature result there.

```
docs/testing/test-runs/<YYYYMMDD-HHMM>-full-audit/
  environment.json
  repo-state.txt
  command-log.tsv
  feature-results.csv
  function-inventory.csv
  feature-entrypoint-map.csv
  existing-test-map.csv
  missing-test-plan.csv
  pass-fail-results.csv
  failures/
  artifacts/
  final-report.md
```

Evidence file	Required content
environment.json	OS, shell, Python/Node/Bun versions, PATH-sensitive tools, network/credential/gate mode
repo-state.txt	git status, branch, commit SHA, remotes, submodule state if any
command-log.tsv	command, cwd, start/end time, exit code, stdout path, stderr path, linked feature IDs
feature-results.csv	one row per atomic feature with status, evidence refs, tests, failure reason
failures/	reproduction commands, stdout/stderr excerpts, expected vs actual, suspected root cause
artifacts/	all generated JSON, logs, screenshots, schema results, installer temp receipts

4. Test report template

Final report must be evidence-linked. The agent should write final-report.md using this structure and include feature IDs from the workbook.

```
# AI4Research Test Report
```

```
## 1. Executive Summary
- Tested repo / branch / commit
- Overall verdict
- Major blockers
- Feature counts: tested / passed / failed / blocked expected / inconclusive / skipped

## 2. Environment
- OS, shell, Python, Node, Bun, git, bash, tmux, jq
- Network mode
- Credential mode
- Gate mode
- Isolated paths used

## 3. Inventory Validation
- Feature rows in spreadsheet
- Functions/scripts/routes discovered
- Missing from spreadsheet
- Stale spreadsheet rows
- Unmapped functions

## 4. Test Coverage Summary
- By part: workflow / foundations / misc
- By feature level
- By automation type
- By priority / risk

## 5. Detailed Feature Results
For each feature ID:
- feature path
- entrypoints
- tests run
- status
- evidence references
- failure reason or gated reason

## 6. Failures and Defects
- P0/P1/P2/P3/P4
- reproduction command
- expected vs actual
- suspected root cause
- affected features

## 7. Gated / Inconclusive Surfaces
- what could not be executed
- why
- required approval/environment/provider

## 8. Missing Tests
- features without direct test
- recommended tests

## 9. Final Verdict
- release readiness
- parity status if relevant
- recommended next actions
```

5. Copy-paste coding-agent prompt

Use the prompt below when giving the workbook and markdown runbooks to a coding agent.

```
You are performing a test audit of the AI4Research repository only. Do not modify production code unless
explicitly requested. Use ai4research_recursive_feature_split_qa_execution.xlsx as the feature taxonomy
and testing control workbook.

Target:
- Repo: https://github.com/Stellven/AI4Research
- Branch: openJiuwen-Solar
- Commit: record exact SHA with git rev-parse HEAD before testing.

Tasks:
```

```
1. Validate the spreadsheet against the current repo checkout.
2. Generate a function/script/route/schema/workflow/package-script inventory.
3. Map every discovered implementation item to an atomic feature ID or mark it support/dead/unmapped.
4. Map every atomic feature to entrypoints, existing tests, missing tests, and pass/fail criteria.
5. Run tests in isolated temp directories. Do not use the user's real ~/.solar or ~/.claude.
6. Do not use real credentials. Do not perform destructive mutations, remote execution, email sending,
release/tag/push/upload, or credential writing.
7. Treat correct approval-gated blocking as BLOCKED_EXPECTED, not as failure.
8. Treat fixture-only, provider-missing, or approval-missing outcomes as INCONCLUSIVE_EXPECTED when the
system reports them honestly through typed evidence.
9. Store all command evidence under docs/testing/test-runs/<timestamp>-full-audit/.
10. Produce final-report.md using the report template in this document.

Required result classifications:
PASS, FAIL, BLOCKED_EXPECTED, INCONCLUSIVE_EXPECTED, SKIPPED_NA, SKIPPED_ENV, FLAKY, NOT_RUN.

Automatic P0/P1 conditions:
- destructive side effect without approval
- credential leak or credential mutation without approval
- remote execution or email send without approval
- release/tag/push/upload without release authorization
- evidence gate accepts malformed or unsupported evidence
- fixture evidence is counted as live/full parity
- installer mutates real user home during isolated test
- /research hides stages inside a black-box backend full workflow

Deliverables:
- final-report.md
- command-log.tsv
- feature-results.csv
- function-inventory.csv
- feature-entrypoint-map.csv
- existing-test-map.csv
- missing-test-plan.csv
- pass-fail-results.csv
- artifacts/failures directory with reproduction evidence.
```